

PYTHON PROJECT REPORT

---- To-Do List Application in Python (Tkinter) – Project File ----

Student Name	Pawandeep kaur
UID	24BSC100008
Section	24BSC-1A
Faculty Name	Ms. Monika
Submission Date	30 Oct 2026

1. Introduction

This project is a desktop To-Do List application written in Python with the Tkinter library. It lets a user add tasks, mark them as done and delete them from a simple window with buttons.

A to-do list is a good first GUI project because it uses the core ideas of programming in a small program: functions, user input, events and a visual interface. Tkinter comes with Python, so nothing needs to be installed.

The whole program is about 60 lines long and is written without classes, so a beginner can read and understand every line.

2. Objectives

1. To build a working GUI application using Python and Tkinter.
2. To practise writing simple functions and connecting them to buttons.
3. To learn how widgets such as labels, entry boxes, buttons and list boxes work.
4. To handle user input and show messages for wrong input.
5. To create a project that a beginner can read, run and extend.

3. Requirements

The program needs only Python and its built-in Tkinter library.

Type	Requirement
Language	Python 3.8 or newer
Library	Tkinter (built in on Windows and macOS)
Editor	IDLE, VS Code, PyCharm or any text editor
Operating system	Windows, macOS or Linux
Hardware	Any computer that can run Python

On some Linux systems Tkinter must be installed once with `sudo apt install python3-tk`.

4. Features and Usage

The application has three features. Tasks are kept only while the window is open.

Feature	Steps
Add a task	Type in the text box, then click **Add Task**
Mark a task done	Click the task in the list, then click **Mark Done** (the task turns

Feature	Steps
	gray)
Delete a task	Click the task in the list, then click **Delete Task**

If the text box is empty, or no task is selected, the program shows a small popup message instead of failing.

5. Design and Working

The program has two parts: functions that do the work, and widgets that build the window.

Functions

Function	What it does
`add_task()`	Reads the text box, adds the text to the list and clears the box. Warns if the box is empty
`delete_task()`	Finds the selected task and removes it
`mark_done()`	Finds the selected task and changes its color to gray

Widgets

Widget	Purpose
`tk.Tk()`	Creates the main window
`tk.Label`	Shows the title text
`tk.Entry`	The box where the user types a task
`tk.Button`	Runs a function when clicked, using `command=`
`tk.Listbox`	Shows the list of tasks
`messagebox`	Shows popup messages
`window.mainloop()`	Keeps the window open and waits for clicks

Working of the program

1. The program creates the window and places the title, text box, buttons and list.
2. `mainloop()` keeps the window open and waits for the user.
3. When the user clicks a button, Tkinter runs the function linked to that button.
4. The function reads or changes the list box and the screen updates at once.

6. Source Code

File name: `simple_todo.py`

```
# Simple To-Do List App (very easy version)
# Run it with:  python simple_todo.py

import tkinter as tk
from tkinter import messagebox

# ---------- Functions (what the buttons do) ----------

def add_task():
    task = entry.get()                      # read what the user typed
    if task == "":
        messagebox.showwarning("Oops", "Please type a task.")
    else:
        listbox.insert(tk.END, task)        # add it to the list
        entry.delete(0, tk.END)             # clear the text box


def delete_task():
    selected = listbox.curselection()       # which task is selected?
    if selected:
        listbox.delete(selected[0])         # remove it
    else:
        messagebox.showinfo("Info", "Click a task first.")


def mark_done():
    selected = listbox.curselection()
    if selected:
        index = selected[0]
        listbox.itemconfig(index, fg="gray")   # turn the task gray
    else:
        messagebox.showinfo("Info", "Click a task first.")


# ---------- Window and widgets ----------

window = tk.Tk()
window.title("My To-Do List")
window.geometry("350x420")

title = tk.Label(window, text="My To-Do List", font=("Arial", 16, "bold"))
title.pack(pady=10)

entry = tk.Entry(window, width=30, font=("Arial", 12))
entry.pack(pady=5)

add_button = tk.Button(window, text="Add Task", width=15, command=add_task)
add_button.pack(pady=5)
```

```
listbox = tk.Listbox(window, width=35, height=10, font=("Arial", 12))
listbox.pack(pady=10)

done_button = tk.Button(window, text="Mark Done", width=15, command=mark_done)
done_button.pack(pady=3)

delete_button = tk.Button(window, text="Delete Task", width=15,
command=delete_task)
delete_button.pack(pady=3)

# Keep the window open
window.mainloop()
```

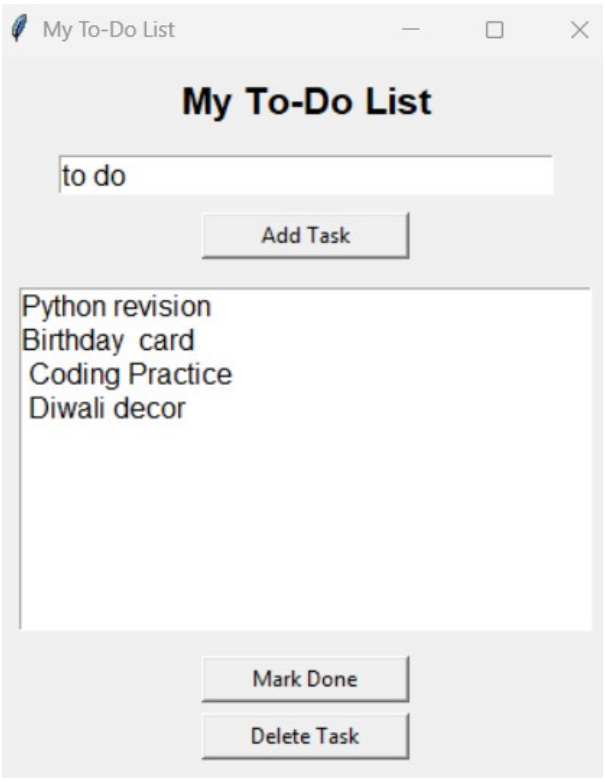
7. How to Run

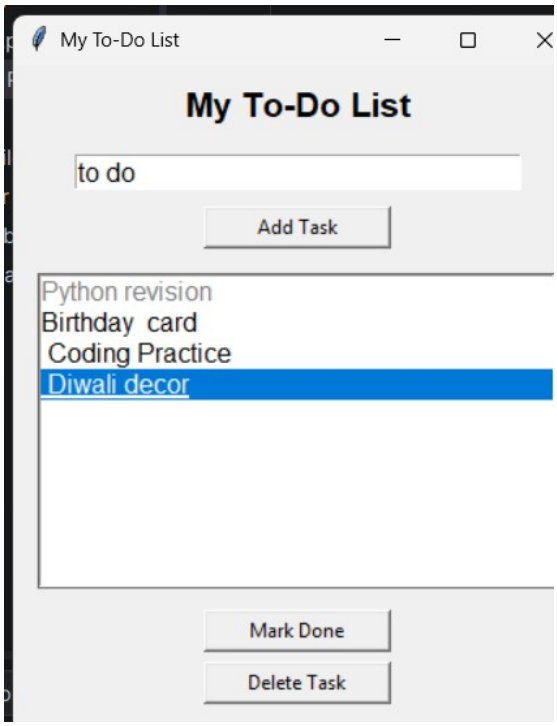
1. Install Python 3.8 or newer from python.org if it is not already installed.
2. Save the code from Section 6 in a file named `simple_todo.py`.
3. Open a terminal or command prompt in the folder that holds the file.
4. Run `python simple_todo.py` (use `python3 simple_todo.py` on macOS or Linux).
5. The To-Do List window opens and is ready to use.

8. Output and Testing

On running the program, a window titled "My To-Do List" opens with a text box, an Add Task button, an empty list and the Mark Done and Delete Task buttons. Run each case below and record the result in the last column.

No.	Test case	Expected result
1	Type "Buy milk" and click Add Task	Task appears in the list and the text box is cleared
2	Click Add Task with an empty text box	Popup: "Please type a task."
3	Select a task and click Mark Done	The task turns gray
4	Select a task and click Delete Task	The task is removed from the list
5	Click Delete Task with no task selected	Popup: "Click a task first."
6	Click Mark Done with no task selected	Popup: "Click a task first."






```
Simple To-Do List App

import tkinter as tk
from tkinter import messagebox

---------- Functions (what the buttons do) ----------

def add_task():  1 usage
    task = entry.get()                    # read what the user typed
    if task == "":
        messagebox.showwarning( title: "Oops",  message: "Please type a task.")
    else:
        listbox.insert(tk.END,  *elements: task)        # add it to the list
        entry.delete( first: 0, tk.END)             # clear the text box


def delete_task():  1 usage
    selected = listbox.curselection()       # which task is selected?
    if selected:
        listbox.delete(selected[0])         # remove it
    else:
```

9. Limitations and Future Scope

Limitations

- Tasks are lost when the window is closed because nothing is saved.
- A task cannot be edited after it is added.
- "Mark Done" only changes the color, and a second click does not undo it.

Future scope

1. Save tasks to a text or JSON file and load them at start-up.
2. Add an Edit Task button and a Clear All button.
3. Let the Enter key add a task.
4. Add due dates and priorities.
5. Add a search box and filters for pending and completed tasks.

10. Conclusion

The To-Do List application shows that a useful GUI program can be written in about 60 lines of Python. It covers windows, widgets, buttons, functions, user input and popup messages.

The project meets its objectives and gives a base that can be extended with file saving, editing and filters. It is a good first step towards larger Tkinter and Python projects.

11. References

1. Python Software Foundation. tkinter: Python interface to Tcl/Tk.
2. Python Software Foundation. The Python Tutorial.
3. Python Software Foundation. tkinter.messagebox documentation.